

57 Array Problems (Complete Master Code)

Pattern 1 – Single Loop (13 Problems)

java

// Group 1: Largest & Smallest Element

```
public class MinMax {
    public static void main(String[] args) {
        int[] arr = {12, 35, 1, 10, 34, 1};
        int max = arr[0], min = arr[0];
        for (int i = 1; i < arr.length; i++) {
            if (arr[i] > max) max = arr[i]; // 1. Largest
            if (arr[i] < min) min = arr[i]; // 2. Smallest
        }
    }
}
```

// Group 2: Second Largest & Second Smallest

```
public class SecondMinMax {
    public static void main(String[] args) {
        int[] arr = {12, 35, 1, 10, 34, 1};
        int max = Integer.MIN_VALUE, secondMax = Integer.MIN_VALUE;
        int min = Integer.MAX_VALUE, secondMin = Integer.MAX_VALUE;
        for (int i = 0; i < arr.length; i++) {
            if (arr[i] > max) {
                secondMax = max;
                max = arr[i];
            } else if (arr[i] > secondMax && arr[i] != max) {
                secondMax = arr[i]; // 3. Second Largest
            }
            if (arr[i] < min) {
                secondMin = min;
                min = arr[i];
            } else if (arr[i] < secondMin && arr[i] != min) {
                secondMin = arr[i]; // 4. Second Smallest
            }
        }
    }
}
```

```
// Group 3: Sum, Average, Count Even & Odd
public class MathOperations {
    public static void main(String[] args) {
        int[] arr = {1, 2, 3, 4, 5};
        int sum = 0, evenCount = 0, oddCount = 0;
        for (int i = 0; i < arr.length; i++) {
            sum += arr[i]; // 5. Sum
            if (arr[i] % 2 == 0) evenCount++; // 7. Count Even
            else oddCount++; // 8. Count Odd
        }
        double average = (double) sum / arr.length; // 6. Average
    }
}
```

```
// Group 4: Search, Count Occurrence, First & Last Occurrence
public class ArraySearches {
    public static void main(String[] args) {
        int[] arr = {10, 20, 30, 20, 40, 20};
        int target = 20, firstIndex = -1, lastIndex = -1, count = 0;
        for (int i = 0; i < arr.length; i++) {
            if (arr[i] == target) { // 9. Linear Search
                if (firstIndex == -1) firstIndex = i; // 11. First Occurrence
                lastIndex = i; // 12. Last Occurrence
                count++; // 10. Count Occurrence
            }
        }
    }
}
```

```
// Group 5: Missing Number
public class MissingNumber {
    public static void main(String[] args) {
        int[] arr = {1, 2, 4, 5, 6};
        int n = 6;
        int expectedSum = (n * (n + 1)) / 2;
        int actualSum = 0;
        for (int num : arr) actualSum += num;
        int missing = expectedSum - actualSum; // 13. Missing Number
    }
}
```

Pattern 2 – Nested Loop (7 Problems)

```
```java
// Group 6: Frequency, Duplicate Elements, Unique Elements
public class ArrayFrequency {
 public static void main(String[] args) {
 int[] arr = {10, 20, 20, 10, 10, 30};
 int n = arr.length;
 boolean[] visited = new boolean[n];
 for (int i = 0; i < n; i++) {
 if (visited[i]) continue;
 int count = 1;
 for (int j = i + 1; j < n; j++) {
 if (arr[i] == arr[j]) {
 count++;
 visited[j] = true;
 }
 }
 // 14. Frequency (count)
 // 15. Duplicate Elements (if count > 1)
 // 16. Unique Elements (if count == 1)
 }
 }
}

// Group 7: Most Frequent & Least Frequent
public class MostLeastFrequent {
 public static void main(String[] args) {
 int[] arr = {10, 20, 20, 10, 10, 30};
 int n = arr.length;
 boolean[] visited = new boolean[n];
 int maxCount = 0, minCount = Integer.MAX_VALUE;
 for (int i = 0; i < n; i++) {
 if (visited[i]) continue;
 int count = 1;
 for (int j = i + 1; j < n; j++) {
 if (arr[i] == arr[j]) { count++; visited[j] = true; }
 }
 if (count > maxCount) maxCount = count; // 17. Most Frequent
 }
 }
}
```

```

 if (count < minCount) minCount = count; // 18. Least Frequent
 }
}
}

// Group 8: Pair With Given Sum & Three Sum
public class TargetSum {
 public static void main(String[] args) {
 int[] arr = {1, 5, 7, -1, 5, 2};
 int pairTarget = 6, threeTarget = 8;
 // 19. Pair With Given Sum
 for (int i = 0; i < arr.length; i++) {
 for (int j = i + 1; j < arr.length; j++) {
 if (arr[i] + arr[j] == pairTarget) { /* Found Pair */ }
 }
 }
 // 20. Three Sum
 for (int i = 0; i < arr.length; i++) {
 for (int j = i + 1; j < arr.length; j++) {
 for (int k = j + 1; k < arr.length; k++) {
 if (arr[i] + arr[j] + arr[k] == threeTarget) { /* Found Triplet */ }
 }
 }
 }
 }
}

```

## Pattern 3 – Sorting / Swap (6 Problems)

java

// Group 9: Classic Sorting (Bubble, Selection, Insertion)

```

public class SortingAlgos {
 public static void main(String[] args) {
 int[] arr = {64, 25, 12, 22, 11};
 int n = arr.length;

 // 21. Bubble Sort
 for (int i = 0; i < n - 1; i++) {
 for (int j = 0; j < n - 1 - i; j++) {
 if (arr[j] > arr[j + 1]) {
 int temp = arr[j]; arr[j] = arr[j + 1]; arr[j + 1] = temp;
 }
 }
 }
 }
}

```

```

 }
}

// 22. Selection Sort
for (int i = 0; i < n - 1; i++) {
 int minIdx = i;
 for (int j = i + 1; j < n; j++) {
 if (arr[j] < arr[minIdx]) minIdx = j;
 }
 int temp = arr[minIdx]; arr[minIdx] = arr[i]; arr[i] = temp;
}

// 23. Insertion Sort
for (int i = 1; i < n; i++) {
 int key = arr[i];
 int j = i - 1;
 while (j >= 0 && arr[j] > key) {
 arr[j + 1] = arr[j]; j = j - 1;
 }
 arr[j + 1] = key;
}
}
}

// Group 10: Swapping Problems (Reverse, Move Zeroes, Sort 0s & 1s)
public class SwapProblems {
 public static void main(String[] args) {
 // 24. Reverse Array
 int[] arr1 = {1, 2, 3, 4, 5};
 int start = 0, end = arr1.length - 1;
 while (start < end) {
 int temp = arr1[start]; arr1[start] = arr1[end]; arr1[end] = temp;
 start++; end--;
 }

 // 25. Move Zeroes to End
 int[] arr2 = {0, 1, 0, 3, 12};
 int nonZeroIdx = 0;
 for (int i = 0; i < arr2.length; i++) {
 if (arr2[i] != 0) {

```

```

 int temp = arr2[i]; arr2[i] = arr2[nonZeroIdx]; arr2[nonZeroIdx] =
temp;
 nonZeroIdx++;
 }
}

```

```

// 26. Sort 0s and 1s
int[] arr3 = {0, 1, 1, 0, 1, 0};
int left = 0, right = arr3.length - 1;
while (left < right) {
 if (arr3[left] == 0) left++;
 else if (arr3[right] == 1) right--;
 else {
 int temp = arr3[left]; arr3[left] = arr3[right]; arr3[right] = temp;
 left++; right--;
 }
}
}
}
}

```

## Pattern 4 – Rotation / Two Pointer (6 Problems)

```

```java
// Group 11: Rotate By One
public class RotateByOne {
    public static void main(String[] args) {
        // 27. Left Rotate by One
        int[] leftArr = {1, 2, 3, 4, 5};
        int tempLeft = leftArr[0];
        for (int i = 1; i < leftArr.length; i++) leftArr[i - 1] = leftArr[i];
        leftArr[leftArr.length - 1] = tempLeft;

        // 28. Right Rotate by One
        int[] rightArr = {1, 2, 3, 4, 5};
        int tempRight = rightArr[rightArr.length - 1];
        for (int i = rightArr.length - 1; i > 0; i--) rightArr[i] = rightArr[i - 1];
        rightArr[0] = tempRight;
    }
}
}

```

```
// Group 12: Rotate By K
public class RotateByK {
    public static void reverse(int[] arr, int start, int end) {
        while (start < end) {
            int temp = arr[start]; arr[start] = arr[end]; arr[end] = temp;
            start++; end--;
        }
    }
    public static void main(String[] args) {
        int[] arr = {1, 2, 3, 4, 5, 6, 7};
        int n = arr.length, k = 3 % n;

        // 29. Right Rotate by K
        reverse(arr, 0, n - 1); reverse(arr, 0, k - 1); reverse(arr, k, n - 1);

        // 30. Left Rotate by K (Opposite order)
        int[] arr2 = {1, 2, 3, 4, 5, 6, 7};
        reverse(arr2, 0, k - 1); reverse(arr2, k, n - 1); reverse(arr2, 0, n - 1);
    }
}
```

```
// Group 13: Two Pointer Merge & Duplicates
public class ClassicTwoPointer {
    public static void main(String[] args) {
        // 31. Remove Duplicates
        int[] dupArr = {1, 1, 2, 2, 3, 4, 4};
        int i = 0;
        for (int j = 1; j < dupArr.length; j++) {
            if (dupArr[i] != dupArr[j]) { i++; dupArr[i] = dupArr[j]; }
        }

        // 32. Merge Two Sorted Arrays (In new Array)
        int[] a = {1, 3, 5}, b = {2, 4, 6};
        int[] merged = new int[a.length + b.length];
        int p1 = 0, p2 = 0, k = 0;
        while (p1 < a.length && p2 < b.length) {
            if (a[p1] < b[p2]) merged[k++] = a[p1++];
            else merged[k++] = b[p2++];
        }
        while (p1 < a.length) merged[k++] = a[p1++];
    }
}
```

```

        while (p2 < b.length) merged[k++] = b[p2++];
    }
}
## Pattern 5 – Sliding Window (3 Problems)
java
// Group 14: Sliding Window Sum
public class SlidingWindowSum {
    public static void main(String[] args) {
        // 33. Maximum Sum of Window Size K
        int[] arr = {1, 4, 2, 10, 2, 3, 1, 0, 20};
        int k = 4, windowSum = 0, maxSum = 0;
        for (int i = 0; i < k; i++) windowSum += arr[i];
        maxSum = windowSum;
        for (int i = k; i < arr.length; i++) {
            windowSum = windowSum + arr[i] - arr[i - k];
            if (windowSum > maxSum) maxSum = windowSum;
        }
    }
}

```

```

// Group 15: Window Traversal
public class WindowTraversal {
    public static void main(String[] args) {
        int[] arr = {12, -1, -7, 8, -15, 30, 16, 28};
        int k = 3, n = arr.length;

        // 34. Maximum Element in Every Window
        for (int i = 0; i <= n - k; i++) {
            int max = arr[i];
            for (int j = i + 1; j < i + k; j++) {
                if (arr[j] > max) max = arr[j];
            }
        }

        // 35. First Negative Number in Every Window
        for (int i = 0; i <= n - k; i++) {
            boolean found = false;
            for (int j = i; j < i + k; j++) {
                if (arr[j] < 0) { found = true; break; }
            }
        }
    }
}

```

```

    }
}
}
## Pattern 6 – Running Sum / Kadane (5 Problems)
java
// Group 16: Target Sums & Equilibrium
public class RunningSumTargets {
    public static void main(String[] args) {
        // 36. Find Subarray With Given Sum
        // 37. Longest Subarray With Given Sum (Same logic, track max
length)
        int[] arr1 = {1, 4, 20, 3, 10, 5};
        int target = 33, currentSum = 0, start = 0;
        for (int i = 0; i < arr1.length; i++) {
            currentSum += arr1[i];
            while (currentSum > target && start <= i) {
                currentSum -= arr1[start]; start++;
            }
            if (currentSum == target) { /* Found */ break; }
        }

        // 38. Equilibrium Index
        int[] arr2 = {-7, 1, 5, 2, -4, 3, 0};
        int totalSum = 0, leftSum = 0;
        for (int num : arr2) totalSum += num;
        for (int i = 0; i < arr2.length; i++) {
            totalSum -= arr2[i];
            if (leftSum == totalSum) { /* Equilibrium found */ break; }
            leftSum += arr2[i];
        }
    }
}

```

```

// Group 17: Kadane's Core
public class KadanesAlgorithm {
    public static void main(String[] args) {
        // 39. Maximum Subarray Sum
        int[] arr3 = {-2, 1, -3, 4, -1, 2, 1, -5, 4};
        int maxSum = arr3[0], sum = 0;
        for (int i = 0; i < arr3.length; i++) {

```

```

        sum += arr3[i];
        if (sum > maxSum) maxSum = sum;
        if (sum < 0) sum = 0;
    }

    // 40. Maximum Product Subarray
    int[] arr4 = {2, 3, -2, 4};
    int n = arr4.length, maxProduct = arr4[0];
    int prefix = 1, suffix = 1;
    for (int i = 0; i < n; i++) {
        if (prefix == 0) prefix = 1;
        if (suffix == 0) suffix = 1;
        prefix *= arr4[i];
        suffix *= arr4[n - i - 1];
        int currentMax = prefix > suffix ? prefix : suffix;
        if (currentMax > maxProduct) maxProduct = currentMax;
    }
}
}
}
...

```

Pattern 7 – Binary Search (3 Problems)

java

// Group 18: Binary Search Master Template

```

public class BinarySearchLogic {
    public static void main(String[] args) {
        int[] arr = {10, 20, 20, 20, 30, 40, 50};
        int target = 20, n = arr.length;

        // 41. Binary Search
        int low = 0, high = n - 1, index = -1;
        while (low <= high) {
            int mid = low + (high - low) / 2;
            if (arr[mid] == target) { index = mid; break; }
            else if (arr[mid] < target) low = mid + 1;
            else high = mid - 1;
        }
    }
}

```

// 42. First Occurrence (Binary Search)

```

low = 0; high = n - 1; int firstOcc = -1;
while (low <= high) {
    int mid = low + (high - low) / 2;
    if (arr[mid] == target) { firstOcc = mid; high = mid - 1; } // Keep
checking left
    else if (arr[mid] < target) low = mid + 1;
    else high = mid - 1;
}

```

```

// 43. Last Occurrence (Binary Search)
low = 0; high = n - 1; int lastOcc = -1;
while (low <= high) {
    int mid = low + (high - low) / 2;
    if (arr[mid] == target) { lastOcc = mid; low = mid + 1; } // Keep
checking right
    else if (arr[mid] < target) low = mid + 1;
    else high = mid - 1;
}
}
}

```

Pattern 8 – Merge (2 Problems)

java

// Group 19: Merge Arrays

```

public class MergeMultipleArrays {
    public static int[] mergeTwoArrays(int[] a, int[] b) {
        int[] merged = new int[a.length + b.length];
        int i = 0, j = 0, k = 0;
        while (i < a.length && j < b.length) {
            if (a[i] < b[j]) merged[k++] = a[i++];
            else merged[k++] = b[j++];
        }
        while (i < a.length) merged[k++] = a[i++];
        while (j < b.length) merged[k++] = b[j++];
        return merged;
    }
}

```

```

public static void main(String[] args) {
    int[] arr1 = {1, 3, 5, 7}, arr2 = {2, 4, 6, 8}, arr3 = {0, 9, 10, 11};
    // 44. Merge Two Sorted Arrays
    int[] resultTwo = mergeTwoArrays(arr1, arr2);
}

```

```

        // 45. Merge Three Sorted Arrays
        int[] tempMerge = mergeTwoArrays(arr1, arr2);
        int[] resultThree = mergeTwoArrays(tempMerge, arr3);
    }
}

## Pattern 9 – Special Interview Algorithms (7 Problems)
java
// Group 20: 46. Trapping Rain Water
public class TrappingRainWater {
    public static void main(String[] args) {
        int[] arr = {3, 0, 2, 0, 4};
        int left = 0, right = arr.length - 1;
        int leftMax = 0, rightMax = 0, water = 0;
        while (left < right) {
            if (arr[left] <= arr[right]) {
                if (arr[left] >= leftMax) leftMax = arr[left];
                else water += leftMax - arr[left];
                left++;
            } else {
                if (arr[right] >= rightMax) rightMax = arr[right];
                else water += rightMax - arr[right];
                right--;
            }
        }
    }
}

// Group 21: 47. Stock Buy and Sell
public class StockBuySell {
    public static void main(String[] args) {
        int[] arr = {7, 1, 5, 3, 6, 4};
        int minPrice = arr[0], maxProfit = 0;
        for (int i = 1; i < arr.length; i++) {
            if (arr[i] < minPrice) minPrice = arr[i];
            else if (arr[i] - minPrice > maxProfit) maxProfit = arr[i] - minPrice;
        }
    }
}

// Group 22: 48. Container With Most Water

```

```

public class ContainerMostWater {
    public static void main(String[] args) {
        int[] arr = {1, 8, 6, 2, 5, 4, 8, 3, 7};
        int left = 0, right = arr.length - 1, maxArea = 0;
        while (left < right) {
            int height = arr[left] < arr[right] ? arr[left] : arr[right];
            int width = right - left;
            if (height * width > maxArea) maxArea = height * width;
            if (arr[left] < arr[right]) left++;
            else right--;
        }
    }
}

```

// Group 23: 49. Leaders in Array

```

public class LeadersInArray {
    public static void main(String[] args) {
        int[] arr = {16, 17, 4, 3, 5, 2};
        int maxFromRight = arr[arr.length - 1];
        // maxFromRight is a leader
        for (int i = arr.length - 2; i >= 0; i--) {
            if (arr[i] > maxFromRight) {
                maxFromRight = arr[i];
                // maxFromRight is a leader
            }
        }
    }
}

```

// Group 24: 50. Majority Element (Boyer-Moore)

```

public class MajorityElement {
    public static void main(String[] args) {
        int[] arr = {2, 2, 1, 1, 2, 2, 2};
        int count = 0, candidate = -1;
        for (int i = 0; i < arr.length; i++) {
            if (count == 0) { candidate = arr[i]; count = 1; }
            else if (arr[i] == candidate) count++;
            else count--;
        }
    }
}

```

```
}
```

```
// Group 25: 51. Longest Consecutive Sequence
```

```
public class LongestConsecutive {  
    public static void main(String[] args) {  
        int[] arr = {100, 4, 200, 1, 3, 2};  
        // Manual Sort (Skipped full Bubble sort code here for brevity)  
        java.util.Arrays.sort(arr);  
        int maxLen = 1, currentLen = 1;  
        for (int i = 1; i < arr.length; i++) {  
            if (arr[i] == arr[i - 1]) continue;  
            if (arr[i] == arr[i - 1] + 1) currentLen++;  
            else { if (currentLen > maxLen) maxLen = currentLen; currentLen = 1; }  
        }  
        if (currentLen > maxLen) maxLen = currentLen;  
    }  
}
```

```
// Group 26: 52. Product of Array Except Self
```

```
public class ProductExceptSelf {  
    public static void main(String[] args) {  
        int[] arr = {1, 2, 3, 4};  
        int n = arr.length;  
        int[] result = new int[n];  
        int leftProduct = 1;  
        for (int i = 0; i < n; i++) {  
            result[i] = leftProduct; leftProduct *= arr[i];  
        }  
        int rightProduct = 1;  
        for (int i = n - 1; i >= 0; i--) {  
            result[i] *= rightProduct; rightProduct *= arr[i];  
        }  
    }  
}
```

```
## Advanced / Optional (5 Problems)
```

```
java
```

```
// 53. Sort 0s, 1s and 2s (Dutch National Flag)
```

```
public class SortColors {  
    public static void main(String[] args) {  
        int[] arr = {2, 0, 2, 1, 1, 0};
```

```

int low = 0, mid = 0, high = arr.length - 1;
while (mid <= high) {
    if (arr[mid] == 0) {
        int temp = arr[low]; arr[low] = arr[mid]; arr[mid] = temp;
        low++; mid++;
    } else if (arr[mid] == 1) {
        mid++;
    } else {
        int temp = arr[mid]; arr[mid] = arr[high]; arr[high] = temp;
        high--;
    }
}
}
}

```

// 54. Rotate Matrix (90°) (Transpose then Reverse Rows)
 // 55. Spiral Matrix Traversal
 // 56. Merge Intervals
 // 57. Set Matrix Zeroes
 // (Matrix and Interval algorithms involve 2D Arrays (int[][])) and require
 specific row/col iterations).

...